

**** LAB ** Automazione con Ansible [14 aprile]**

Sito: [Virtuale](#)

Corso: Amministrazione di Sistemi T - Laboratorio di
Amministrazione di Sistemi T

Libro: ** LAB ** Automazione con Ansible [14 aprile]

Stampato da: Lorenzo Bernardini

Data: venerdì, 24 aprile 2026, 11:54

Descrizione

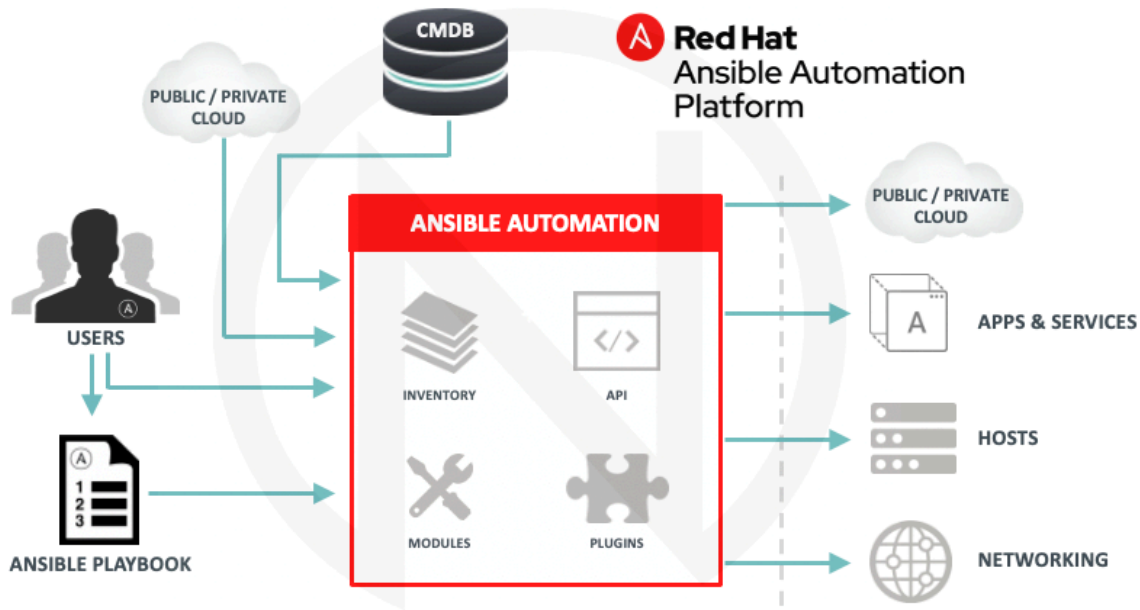
<https://docs.ansible.com/>

Sommario

1. Ansible in a nutshell
2. Concetti chiave
3. Validazione della sintassi
4. Ansible idempotenza dei playbook
5. Installazione
6. Integrazione con Vagrant
7. Esercizio - aggiungere un nuovo utente
8. Esercizio - login con chiave
9. Esercizio - installazione di uno script
10. Esercizio - modifica di un file
11. Esercizio - pianificazione periodica
12. Esercizio - gestione di systemd
13. Esercizio - Handlers

1. Ansible in a nutshell

Ansible è un sistema per automatizzare la gestione di asset quali applicazioni, host e reti.



Source: <https://www.openvirtualization.pro/wp-content/uploads/2021/03/4.png>

2. Concetti chiave

https://docs.ansible.com/ansible/latest/user_guide/basic_concepts.html#basic-concepts

Ansible è un programma che gira su di un **CONTROL NODE**, si collega (via SSH) ai **MANAGED NODES** (che non necessitano di nulla più del server SSH) e vi esegue dei **TASK** definiti in un **PLAYBOOK**.

La gestione di molteplici Managed Nodes può essere agevolata creando degli **INVENTORY** nei quali elencarli ed eventualmente riunirli in gruppi, anche annidati.

È possibile definire **ROLES** che permettono di raggruppare tutti i task, le variabili, e le dipendenze comuni da applicare a un host o gruppo che deve essere configurato per ottenere una finalità ripetibile, in modo che il Playbook risulti più conciso e leggibile.

I Task sono svolti per mezzo di **MODULES**, che raggruppano tutte le funzioni utili per un determinato sottosistema (ad esempio la gestione degli utenti, o dei pacchetti software, ecc.) astruendo dai dettagli del Managed Node (o viceversa consentendo di sfruttare specifiche potenzialità di determinati sistemi operativi). Nelle versioni più recenti (2.10 e successive) i Modules sono raggruppati in **COLLECTIONS**.

La miglior pratica consigliata prevede di articolare le informazioni in un insieme di file e directory ordinato, ad **esempio**:

```
production      # inventory file for production servers
staging         # inventory file for staging environment

group_vars/
  group1        # here we assign variables to particular groups
  group2        # ""
host_vars/
  hostname1     # if systems need specific variables, put them here
  hostname2     # ""

library/        # if any custom modules, put them here (optional)
module_utils/   # if any custom module_utils to support modules, put them here (optional)
filter_plugins/ # if any custom filter plugins, put them here (optional)

site.yml        # master playbook
webservers.yml  # playbook for webserver tier
dbservers.yml   # playbook for dbserver tier

roles/
  common/       # this hierarchy represents a "role"
    tasks/      #
      main.yml   # <-- tasks file can include smaller files if warranted
    handlers/   #
      main.yml   # <-- handlers file
    templates/  # <-- files for use with the template resource
      ntp.conf.j2 # <----- templates end in .j2
    files/      #
      bar.txt    # <-- files for use with the copy resource
      foo.sh     # <-- script files for use with the script resource
    vars/       #
      main.yml   # <-- variables associated with this role
    defaults/   #
      main.yml   # <-- default lower priority variables for this role
    meta/       #
      main.yml   # <-- role dependencies
    library/    # roles can also include custom modules
    module_utils/ # roles can also include custom module_utils
    lookup_plugins/ # or other types of plugins, like lookup in this case

  webtier/      # same kind of structure as "common" was above, done for the webtier role
  monitoring/   # ""
  fooapp/       # ""
```

3. Validazione della sintassi

I file di Ansible sono scritti in YAML. È utile disporre di uno strumento per validarne la sintassi, che è semplice ma molto sensibile alla posizione di tutti i caratteri di controllo.

Per questo è stato sviluppato [ansible-lint](#)

Es:

```
ansible-lint -vvv playbook.yml
```

4. Ansible idempotenza dei playbook

L'idempotenza è la proprietà di alcune operazioni, che applicate una o più volte ad un sistema non modificano il risultato oltre l'applicazione iniziale.

In Ansible, l'idempotenza è offerta come funzionalità integrata di molti dei moduli Ansible. Ciò si riferisce direttamente al funzionamento di Ansible in base allo stato del managed node. In termini semplici, ciò significa che quando eseguiamo un'attività, vengono apportate le modifiche richieste allo stato del managed node e qualsiasi ulteriore ripetizione di questa attività sul managed node non comporta alcun cambiamento nello stato.

Bisogna quindi limitare al massimo tutte quelle operazioni durante la scrittura dei playbook che possono portare ad una mancanza di idempotenza al termine del processo di provisioning es: utilizzare il modulo `shell`, `command`, `raw` al fine di cancellare un file, la prima volta viene cancellato ma la seconda volta viene ritornato un errore e quindi interrotto a default il provisioning tramite Ansible

5. Installazione

Utilizzeremo l'host Linux come Control Node, e le VM Vagrant come Managed Nodes.

I pacchetti di cui avrete bisogno sul host sono i seguenti, nei pc del lab sono già presenti e quindi non c'è bisogno del seguente comando:

```
sudo apt install ansible ansible-lint ansible-doc
```

A questo punto è possibile usare l'host come Control Node per qualsiasi macchina raggiungibile via SSH su cui sia installato Python, lanciando manualmente il playbook desiderato.

6. Integrazione con Vagrant

Per integrare Ansible con Vagrant, è sufficiente specificarlo nel **Vagrantfile** come metodo di provisioning, indicando il nome del playbook da eseguire:

```
# NOTA: questa riga (e l'end corrispondente) ci sono già nel Vagrantfile
# a meno che non lo stiate scrivendo a mano da zero, nel qual caso mancano
# dovrete specificare a mano anche config.vm.box (ad es. = "debian/bullseye64")
Vagrant.configure("2") do |config|
#
# Run Ansible from the Vagrant Host
#
  config.vm.provision "ansible" do |ansible|
    ansible.playbook = "playbook.yml"
  end
end
```

Poiché il playbook viene applicato alla VM definita dal Vagrantfile, il file **playbook.yml** (che deve essere nella stessa directory) in prima battuta può ignorare il concetto di inventory, dichiarando di voler funzionare per tutte le macchine note (**hosts: all**). Inoltre per ora ci concentriamo su task che richiedono i privilegi di root (**become: true**) Il template da cui partire sarà quindi:

```
---
- name: "Sample playbook" # Ogni playbook deve avere un nome; tutti i "name" devono iniziare con una lettera maiuscola
  hosts: all
  become: true # esegue i comandi remoti con sudo; invece che globale potrebbe essere locale ai task che lo richiedo
```

Note importanti sul [provisioning](#):

- il provisioning avviene automaticamente al primo avvio della VM con **vagrant up**
- Vagrant memorizza di averlo effettuato, per cui ai successivi avvii non lo ripete
- Si può forzare all'avvio con l'opzione `--provision` (**vagrant up --provision**)
- Si può eseguire manualmente su di una VM running col comando **vagrant provision**

7. Esercizio - aggiungere un nuovo utente

Aggiungere ai task

```
- name: "Add the user 'johnd'"
  ansible.builtin.user:
    name: johnd
    comment: John Doe
```

Il trattino davanti a name deve essere indentato rispetto a "tasks:"

[Versione con tutte le spaziature corrette](#)

Estensione proposta:

studiando la documentazione del modulo [user](#) (oppure da linea di comando: `ansible-doc user`) , modificare il task per

- impostare una password di vostra scelta
- disabilitare l'utente dopo 180 giorni

Inoltre,

- aggiungere un [task che crei un ulteriore gruppo](#) "team"
- modificare il task originale per [aggiungere](#) l'utente johnd a team

8. Esercizio - login con chiave

Fare in modo che il vostro utente standard del Control Node possa accedere come "johnd" sul Managed Node, senza bisogno di inserirne la password.

Preliminare: creare una nuova chiave ssh per l'utente del Control Node

```
cd ~/.ssh
```

```
ssh-keygen -t rsa -b 2048 -f id_rsa_ansible
```

(confermare le richieste)

Domanda: ricordando la modalità specifica con cui ci si connette a una VM Vagrant, come si dovrebbe procedere manualmente?

Il task può essere svolto con estrema facilità se è installata la collection [ansible.posix](#)

e si chiama semplicemente [authorized_key](#)

```
- name: Upload ssh public key in Authorized_key
  ansible.posix.authorized_key:
    user: johnd
    state: present
    key: "{{ lookup('file', '~/.ssh/id_rsa_ansible.pub') }}"
```

[Playbook completo](#)

Qualora il modulo `ansible.posix` non dovesse essere disponibile potete utilizzare il modulo [builtin.copy](#) ed il modulo [builtin.file](#)

9. Esercizio - installazione di uno script

Esercizio preliminare:

realizzare sull'host nella vostra home uno script **copy.sh** che crei un archivio di nome **bck.timestamp_in_formato_ISO8601.tgz** contenente tutti i file elencati nel file di configurazione **save.list**

NOTA: da questo esercizio in poi, rimuovere dal playbook la direttiva globale **become: true**, inserendola solo nei task che richiedono i privilegi di amministratore.

Creare un task che copi sulla VM nella home di *vagrant* lo script e il file di configurazione, settando opportunamente i permessi per la sua esecuzione. Si può realizzare con il builtin [copy](#).

```
- name: Copy script
  ansible.builtin.copy:
    src: /path/della/vostra/home/copy.sh
    dest: /home/vagrant/copy.sh
    mode: '0755'
    owner: vagrant
    group: vagrant
```

```
- name: Copy config file
  ansible.builtin.copy:
    src: /path/della/vostra/home/save.list
    dest: /home/vagrant/save.list
    mode: '0644'
    owner: vagrant
    group: vagrant
```

La copia di più file può essere semplificata con un loop (notate che in questo caso non si possono differenziare i permessi, **si dovrebbe fare con un altro task** - [provate](#)):

```
- name: Copy file with owner and permissions
  ansible.builtin.copy:
    src: "/path/della/vostra/home/{{ item }}"
    dest: /home/johnd/
    mode: '0755'
    owner: johnd # l'assegnazione utente richiede become: true
    group: johnd # l'assegnazione gruppo richiede tipicamente become: true
  loop:
    - copy.sh
    - save.list
```

Estensione proposta:

copiare lo script invece che nella home di *vagrant* in **/usr/bin**, il file di configurazione in **/etc**, renderli di proprietà di *root* ed eseguibile solo da *root*

10. Esercizio - modifica di un file

Aggiungere nel playbook il controllo che nel file **save.list** presente sulla VM esista la riga **/etc/passwd**, o in caso contrario inserirla.

Si può utilizzare il builtin [lineinfile](#):

```
- name: Ensure passwd is backed up
  ansible.builtin.lineinfile:
    path: /etc/save.list
    line: /etc/passwd
```

Estensione proposta 1:

garantire che sia assente/rimossa dal file qualsiasi riga inizi con **/etc/passwd**, **/etc/shadow** o **/etc/group**

Estensione proposta 2:

inserire l'utente **johnd** creato in precedenza nei sudoers in modo che possa usare sudo senza password (usate validate)

11. Esercizio - pianificazione periodica

Configurare **cron** per eseguire come root lo script ogni 2 minuti nei giorni feriali escluso agosto.

Anche in questo caso esiste un builtin dedicato [cron](#):

```
- name: Schedule backup
  ansible.builtin.cron:
    name: "backup selected files"
    minute: "*/2"
    weekday: "1-5"
    month: "1-7,9-12"
    job: "/usr/bin/copy.sh 2>/dev/null >/dev/null"
```

Esercizio proposto:

rimuovere il job precedentemente inserito

12. Esercizio - gestione di systemd

Il builtin [systemd](#) si occupa della gestione delle unit, limitatamente al loro avvio o arresto.

Esempio: garantire che il sistema di logging sia attivato

```
- name: Guarantee rsyslog is started
  ansible.builtin.systemd:
    state: started
    name: rsyslog.service
```

Esercizio proposto:

[installare](#) e garantire l'avvio di atd

13. Esercizio - Handlers

Gli [handler](#) sono un modo per far eseguire un task in funzione dell'esecuzione di un altro task.

Ad esempio, potrei desiderare che sia riavviato un demone se il suo file di configurazione viene cambiato.

```
---
- name: Verify apache installation
  hosts: webserver
  vars:
    http_port: 80
    max_clients: 200
  remote_user: root
  tasks:
    - name: Ensure apache is at the latest version
      ansible.builtin.yum:
        name: httpd
        state: latest

    - name: Write the apache config file
      ansible.builtin.template:
        src: /srv/httpd.j2
        dest: /etc/httpd.conf
      notify:
        - Restart apache

    - name: Ensure apache is running
      ansible.builtin.service:
        name: httpd
        state: started

  handlers:
    - name: Restart apache
      ansible.builtin.service:
        name: httpd
        state: restarted
```

Esercizio proposto:

Un caso tipico è quello dell'installazione o dell'aggiornamento di uno unit file di systemd, che richiede l'esecuzione dell'operazione **daemon_reload** di systemd stesso per mezzo del relativo [modulo](#). Realizzare un playbook che copia uno unit file (usando uno tra quelli già esistenti) e testare facendo modifiche ininfluenti (commenti, spazi).